

Modèle objet de document

Aperçu

Une compétence essentielle pour tout développeur web front-end est la capacité à manipuler le [DOM \(Document Object Model\)](#), un objet JavaScript créé par le navigateur après l'analyse du document HTML. Manipuler le DOM signifie lire, modifier, mettre à jour ou supprimer dynamiquement des éléments et leurs propriétés CSS. Le DOM est la représentation arborescente de la structure et du contenu de votre page.

L'objectif de cette activité est de présenter le DOM HTML et d'apprendre à manipuler le document à l'aide de JavaScript.

Résultats d'apprentissage associés au cours

Démontrer une maîtrise de la syntaxe du langage JavaScript.
Utiliser JavaScript pour réagir à des événements et modifier dynamiquement le HTML.

Préparer

Vidéo de présentation : [Le DOM JavaScript expliqué en 5 minutes!](#) – Bro Code

- Référence : [Manipulation de documents](#) – MDN.
Notez que l'exemple [de liste de courses dynamique « Apprentissage actif »](#) que vous trouverez à la fin de l'article MDN vous **aidera** à réaliser les activités de cette semaine.
- Exercices : **Concepts et cas d'utilisation courants de la manipulation du DOM**.
 - **Sélectionnez** un élément HTML du document à l'aide de la méthode [querySelector](#). Cette ligne de code sélectionne la première occurrence d'un élément article dans le document et l'assigne comme référence à la variable nommée **article**.
 - `const article = document.querySelector('article');`
 - **Modifiez** la propriété innerHTML d'un élément existant. Cette ligne de code utilise une variable existante qui référence

un élément précédemment sélectionné et modifie la valeur de sa propriété innerHTML.

- `article.innerHTML = 'innerHTML supports <strong>HTML</strong> tags. The textContent property does not.';`
- **Modifie** le style CSS en ligne d'un élément.
Cette ligne de code modifie la propriété CSS text-align de l'élément sélectionné.
- `article.style.textAlign = 'right';`
- **Modifier** un attribut d'un élément.
Cette ligne de code ajoute un attribut de classe à l'élément et lui attribue une valeur.
- `article.setAttribute('class', 'active');`
- Une autre façon de modifier l'attribut de classe d'un élément consiste à manipuler directement la **classList** propriété de l'élément.
- `article.classList.add('active');`
- Cette méthode est souvent privilégiée pour la manipulation des classes car **classList** elle offre des méthodes supplémentaires comme **add**, **remove**, **toggle**, et **contains**, ce qui la rend plus pratique et expressive pour la gestion des classes.
- **Créer** un élément.
Cette ligne de code crée un nouvel élément <p> et le stocke dans la variable nommée **paragraph**.
- `const paragraph = document.createElement('p');`
- **Ajouter** du contenu ou des éléments à un élément.
Ces lignes de code ajoutent du contenu à la fin de l'**article** élément.

```
article.appendChild(paragraph);
```

- `article.append(paragraph, 'Hello World Addition!');`
- La méthode append() vous permet d'inclure plusieurs arguments à ajouter à l'élément dans l'ordre spécifié.
- **Supprimer** un élément du document.
Ces lignes de code suppriment l' **paragraph** élément du document **article**, puis suppriment l' **article** élément lui-même.

```
article.removeChild(paragraph);
```

- `article.remove();`

Introduction au JavaScript DOM



Niveau : Débutant



Chapitre 1 : Comprendre le lien entre HTML et JavaScript

Lorsque tu crées une page web, tu utilises **HTML** pour afficher des éléments comme :

- un champ de texte
- un bouton
- une liste

Mais HTML seul ne suffit pas pour rendre la page **interactive**.

👉 C'est là que **JavaScript** intervient.

JavaScript permet :

- de lire ce que l'utilisateur écrit
 - de réagir aux clics
 - de modifier la page en temps réel
-

♦ Accéder aux éléments HTML

Dans ton code :

```
const input = document.querySelector("#chapterInput");  
const bouton = document.querySelector("#addBtn");  
const list = document.querySelector("#chapterList");
```



Explication

- `document` représente toute la page
- `querySelector()` permet de chercher un élément

👉 Exemple :

- `#chapterInput` → un champ texte
 - `#addBtn` → un bouton
 - `#chapterList` → une liste
-

Idée importante

Avant de modifier un élément, il faut **le récupérer**.

Chapitre 2 : Réagir aux actions de l'utilisateur

Une page web devient interactive lorsqu'elle réagit aux actions :

- cliquer
 - écrire
 - déplacer la souris
-

♦ Écouter un clic

```
bouton.addEventListener("click", () => {
```

Explication

- `"click"` = événement (action)
- `addEventListener` = écouter cet événement
- `() => {}` = fonction exécutée

👉 Cela veut dire :

“Quand l'utilisateur clique sur le bouton, exécute ce code.”

♦ Lire ce que l'utilisateur écrit

```
let verset = input.value;
```

👉 `value` = contenu du champ texte

♦ Vérifier si c'est vide

```
if (verset === ""){  
    alert("Veuillez entrer chapitre avant de l'ajouter à la liste.");  
    return;  
}
```

🧠 Explication

- `if` = condition
 - `alert()` = afficher un message
 - `return` = arrêter le programme
-

🎯 Idée importante

Toujours vérifier les données de l'utilisateur avant de les utiliser.

📖 Chapitre 3 : Créer des éléments dynamiques

JavaScript permet de **créer des éléments HTML directement**.

♦ Créer un élément

```
let chapitre = document.createElement("li");
```

👉 Cela crée un élément `<li>` (élément de liste)

♦ Ajouter du texte

```
chapitre.textContent = verset;
```

👉 On met le texte dedans

◆ Créer un bouton supprimer

```
let supbtn = document.createElement("button");  
supbtn.textContent = "✖";
```

👉 Un bouton avec une croix

◆ Ajouter une action au bouton

```
supbtn.addEventListener("click", ()=> {  
  chapitre.remove();  
})
```

👉 Quand on clique :
➡ l'élément est supprimé

◆ Assembler les éléments

```
chapitre.appendChild(supbtn)  
list.appendChild(chapitre);
```

👉 Résultat :

- le bouton est dans le **li**
 - le **li** est dans la liste
-

◆ Nettoyer le champ

```
input.value = "";
```

👉 On vide le champ après ajout

Idée importante

JavaScript peut :

- créer
- modifier

- supprimer des éléments

👉 en direct dans la page

Chapitre 4 : Styliser avec JavaScript

JavaScript peut aussi modifier le **design**.

◆ Sélection du footer

```
let footer = document.querySelector("footer");
```

◆ Ajouter du texte

```
footer.textContent = "© 2024, meschapitre preferer - Tous droits réservés.";
```

◆ Modifier le style

```
footer.style.textAlign = "center";  
footer.style.padding = "10px";  
footer.style.backgroundColor = "#bc1212";  
footer.style.position = "fixed";  
footer.style.bottom = "0";  
footer.style.left = "0";  
footer.style.color = "#f0eeee";  
footer.style.width = "100%";
```

Explication simple

- `textAlign` → centrer
 - `backgroundColor` → couleur
 - `position: fixed` → reste collé
 - `bottom: 0` → en bas
-

Résultat

👉 Le footer reste toujours visible en bas de la page.



Chapitre 5 : Résumé général

Ton programme permet de :

- ✓ Lire ce que l'utilisateur écrit
 - ✓ Ajouter un élément dans une liste
 - ✓ Supprimer un élément
 - ✓ Mettre à jour la page sans recharger
 - ✓ Styliser une partie de la page
-



Conclusion du livre

Avec ce projet simple, tu as appris les bases du **DOM (Document Object Model)** :

- sélectionner des éléments
- écouter des événements
- créer du contenu
- manipuler la page
-